

۱. بخش الویت اولویت عملگرها و روش استفاده شده برای رفع dangling else را توضیح دهید.

برای رفع dangling else گرامر مربوط به این بخش به صورت زیر نوشته شده است:

```
stmt : decide(' expr ') stmt stm_decide_prime ;
```

```
stm_decide_prime : ('else' stmt)?
```

```
|
```

```
;
```

به وسیله ؟ ('else' stmt) بررسی می شود که آیا بعد از این شرط else وجود دارد یا نه و اگر باشد، نزدیک ترین else به آن if برای آن در نظر گرفته می شود.

برای رعایت الویت بندی، گرامر مربوط به این بخش به صورت زیر نوشته شده است:

```
expr : unop expr
```

```
| expr binop expr ;
```

```
binop : ( '*' | '/' | '%' )
```

```
| ( '+' | '-' )
```

```
| ( '<' | '>' | '<=' | '>=' )
```

```
| ( '==' | '!=' )
```

```
| and
```

```
| or
```

```
;
```

```
and : '&&' ;
```

```
or : '||' ;
```

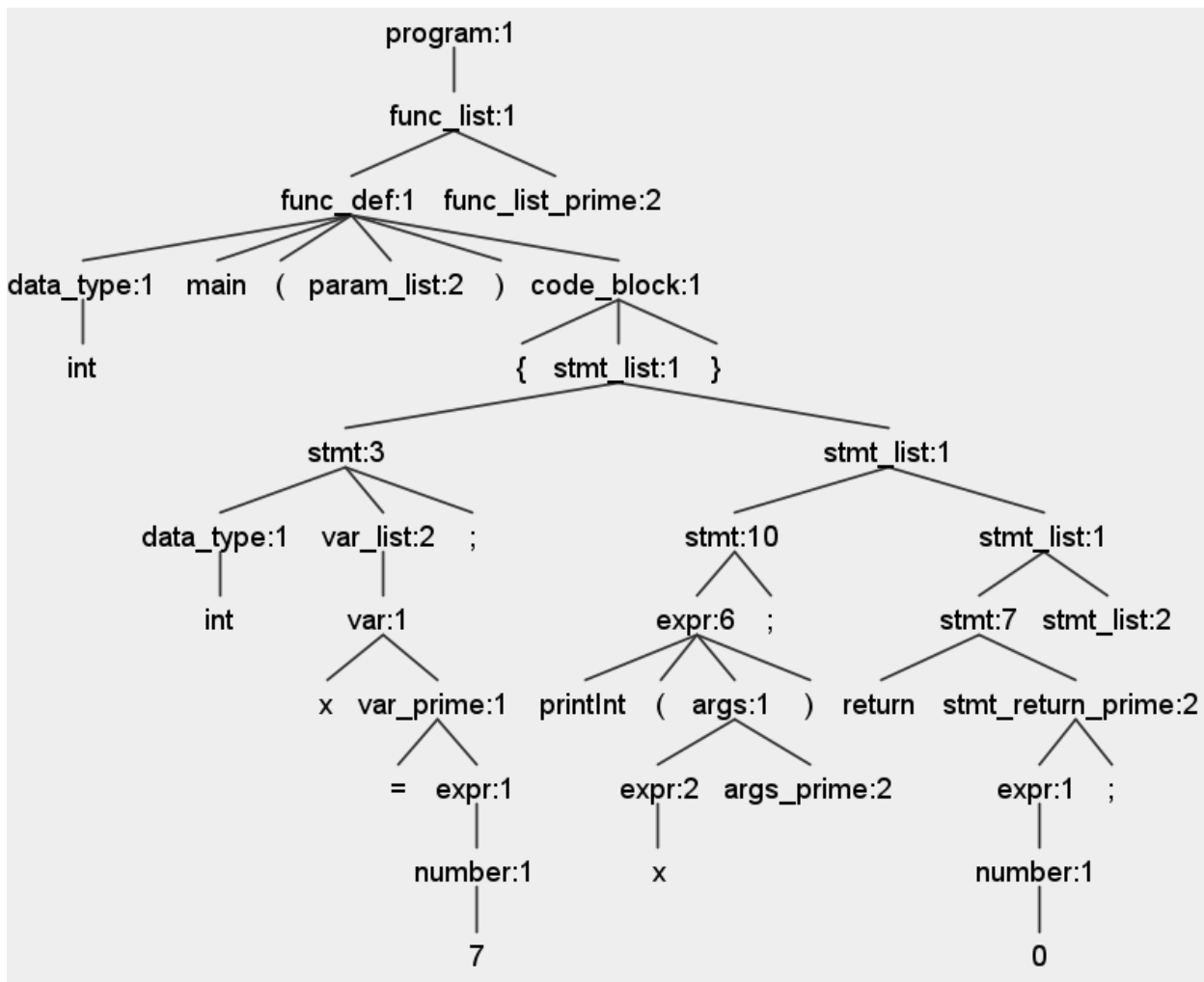
برای رعایت الویت بندی، عملگرها به ترتیب الویت نوشته شده اند و ابتدا unary و بعد باینری ها به ترتیب الویت نوشته شده پردازش می شوند. برای عملگرهای دودویی هم ابتدا عملگرهای محاسباتی و سپس تساوی و سپس عملگرهای منطقی مورد پردازش قرار می گیرند.

۲. خروجی دو درخت تجزیه شده را به دلخواه انتخاب کنید و به صورت خلاصه توضیح دهید.

ورودی ۰۰۷ :

```
3 int main() {  
4   int x = 7;  
5   printInt(x);  
6   return 0 ;  
7  
8 }
```

خروجی :



همانطور که در خروجی مشاهده می‌کنیم، ابتدا تابع تجزیه شده و در بدنه آن که code_block است، شاهد تعریف متغیر x و مقداردهی آن هستیم. همچنین کال شدن تابع هم به درستی تجزیه شده و x به

عنوان ورودی داده شده است. در خوشه آخر تجزیه، شاهد تجزیه return هستیم که مقداری که برمی گرداند، صفر است.

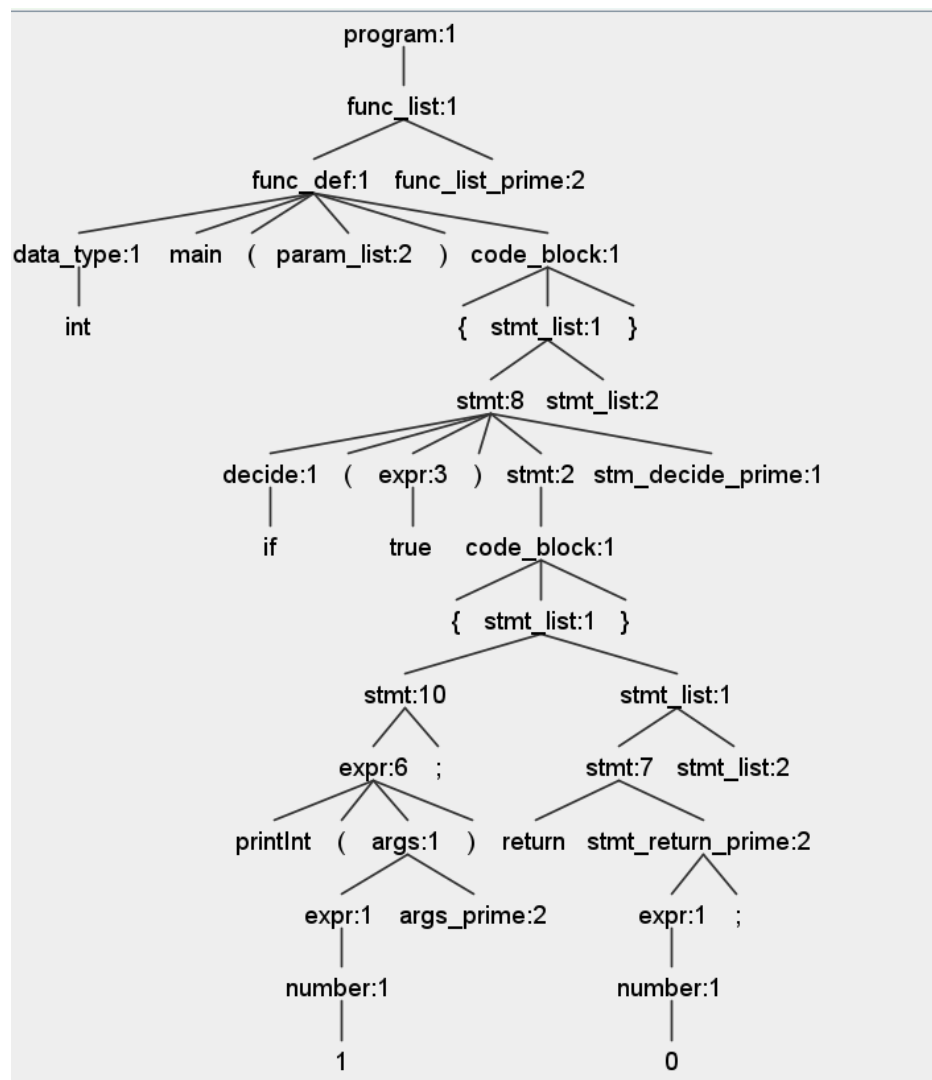
ورودی ۰۲۱ :

```

1  int main() {
2      if (true) {
3          printInt(1);
4          return 0;
5      }
6  }

```

خروجی :



همانطور که مشاهده می‌کنیم، تجزیه با تابع main آغاز می‌شود و سپس وارد شرط می‌شویم و true هم به عنوان شرط قرار گرفته است. چون else نداریم stm_decide_prime برابر اپسیلون می‌شود. در بدنه شرط هم مشاهده می‌کنیم، تابع printInt با ورودی ۱ تجزیه شده است و ۰ هم در انتها برگردانده شده است.

۳. فایل‌های تولیدشده توسط antlr را به صورت خلاصه توضیح دهید.

با اجرای کامند `antlr4 -o src -Dlanguage=Python3 MyGrammer.g4` تعدادی فایل تولید می‌شود:

فایل `MyGrammerLexer.py` برای شکستن ورودی به تعدادی توکن است که هر توکن مربوط به یک rule در گرامر نوشته شده است.

فایل `MyGrammerParser.py` برای توکن‌های تولید شده توسط lexer را می‌گیرد و با توجه به گرامر تعریف شده، یک parse tree ایجاد می‌کند.

فایل `MyGrammerListener.py` به ازای هر non-terminal یک تابع enter و یک تابع exit دارد و نحوه پیمایش درخت را به وسیله این فایل می‌توان تعریف کرد به این صورت که با ورود به هر node و یا خروج از آن چه اتفاقاتی باید بیفتد.

فایل `MyGrammer.tokens` حاوی تمام توکن‌های تشخیص داده شده توسط lexer و parser است.

فایل `MyGrammer.interp` حاوی Serialized ATN است که parser از آن برای parse کردن استفاده می‌کند و به بهینه‌تر انجام شدن این فرایند کمک می‌کند.